

Tuyển Tập Bài Tập: Kỹ Thuật Đảo Nhãn Quy Hoạch Động (DP State-Value Inversion)

(16 Bài toán Tuyển chọn Phân cấp từ AtCoder, Codeforces, CSES & Luogu)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Kỹ thuật Phối hợp	Độ phức tạp Mục tiêu
1	AtCoder DP Task E (Knapsack 2)	Educational DP Contest	Khởi động	Đảo vai trò Trọng lượng $\leftrightarrow$ Giá trị	$O(N \cdot \sum V)$
2	CSES 1145 (Increasing Subsequence)	CSES Problemset	Khởi động	Đảo nhãn $dp[len] = \min tail + \text{Chặt nhị phân}$	$O(N \log N)$
3	Codeforces 1042B (Vitamins)	Codeforces Round 510	Khởi động	$dp[mask] = \min cost$ với $mask$ 3 bits $\{A, B, C\}$	$O(N \cdot 2^3)$
4	Codeforces 1437E (Make It Increasing)	Educational Round 97	Khởi động	Biến đổi $A'[i] = A[i] - i$ + LIS có ràng buộc cố định	$O(N \log N)$
5	Codeforces 864E (Fire)	Codeforces Round 436	Trung cấp	Sắp xếp theo Deadline d_i + Knapsack đảo thời gian	$O(N \cdot \max D)$
6	Codeforces 1221D (Make The Fence Great Again)	Educational Round 73	Trung cấp	Đảo nhãn mức tăng $+0, +1, +2$ tối thiểu chi phí	$O(N)$
7	AtCoder ABC 275 F (Erase Subarrays)	AtCoder Beginner Contest 275	Trung cấp	$dp[x][0/1] = \text{số đoạn xóa ít nhất để tổng bằng } x$	$O(N \cdot M)$
8	Codeforces 437C (The Child and Toy)	Codeforces Round 250	Trung cấp	Đảo ngược góc nhìn: Tính đóng góp của từng cạnh	$O(M)$

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Kỹ thuật Phối hợp	Độ phức tạp Mục tiêu
9	Codeforces 730J (Bottles)	ACM-ICPC NEERC 2016	Nâng cao	Tìm min k chai, rồi $dp[k][cap] = \max$ nước ban đầu	$O(N^2 \cdot \sum B)$
10	Codeforces 19B (Checkout Assistant)	Codeforces Beta Round 19	Nâng cao	Quy về Knapsack: "Trọng lượng" $t_i + 1$, min chi phí $\sum c_i$	$O(N^2)$
11	Codeforces 938D (Buy a Ticket)	Educational Round 38	Nâng cao	Đảo nhãn đồ thị: Tạo siêu nguồn kết nối chi phí vé A_u	$O((N + M) \log N)$
12	AtCoder ABC 221 G (Jumping Frog)	AtCoder Beginner Contest 221	Nâng cao	Xoay hệ trục 45° + Subset Sum đảo nhãn Bitset	$O(N \cdot \frac{\sum D}{64})$
13	Luogu P3188 (【HNOI2007】 邦邦的大背包)	HNOI 2007 / Luogu	Chinh phục	Ba lô $W \leq 10^9$ có dạng $a \cdot 2^b$ + DP phân rã bit nhị phân	$O(\log_2 W \cdot N^2)$
14	Codeforces 1442D (Sum of Paths)	Codeforces Round 681	Chinh phục	Bổ đề Knapsack mảng lồi + Chia để trị loại trừ 1 mảng	$O(N \cdot K \log N)$
15	AtCoder AGC 020 C (Median Sum)	Grand Contest 020	Chinh phục	Đổi ngẫu tập bù $S \leftrightarrow U \setminus S$ + Bitset tối ưu	$O(\frac{N \cdot \sum A}{64})$
16	Codeforces 1083E (The Fair Nut and Rectangles)	Codeforces Round 526	Chinh phục	Đảo nhãn thứ tự hình chữ nhật + Convex Hull Trick	$O(N \log N)$

1. Cấp Độ 1: Khởi Động & Nền Tảng (Warm-up & Fundamentals)

Bài 1: [AtCoder DP Task E - Knapsack 2](#)

- **Nguồn:** [Educational DP Contest — Task E](#).
- **Giới hạn:** $N \leq 100, W \leq 10^9, v_i \leq 1000 \implies \sum v_i \leq 10^5$, Thời gian: 2.0s, Bộ nhớ: 1024 MB.
- **Tóm tắt bài toán:** Chọn một tập các đồ vật sao cho tổng trọng lượng $\leq W$ và tổng giá trị là lớn nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Vì $W \leq 10^9$, mảng `dp[W]` là bất khả thi. Nhưng $\sum v_i \leq 10^5$ rất nhỏ $\implies$ Đảo nhãn trạng thái.
- **Bậc 2 (Thiết kế trạng thái):** Gọi `dp[v]` là **trọng lượng nhỏ nhất** cần dùng để đạt được đúng tổng giá trị v .
- Khởi tạo: `dp[0] = 0`, các ô khác $= \infty$.
- Chuyển trạng thái: Duyệt ngược v từ $\sum v$ về v_i : `dp[v] = min(dp[v], dp[v - v_i] + w_i)`.

- **Bậc 3 (Trích xuất đáp án):** Duyệt v từ $\sum v$ về 0, giá trị đầu tiên thỏa $dp[v] \leq W$ là đáp án. Độ phức tạp $O(N \cdot \sum V)$.

Bài 2: [CSES 1145 - Increasing Subsequence](#)

- **Nguồn:** [CSES Problemset](#).
- **Giới hạn:** $N \leq 2 \times 10^5$, $X_i \leq 10^9$, Thời gian: 1.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Tìm độ dài của dãy con tăng nghiêm ngặt dài nhất (LIS) của mảng gồm N số nguyên.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Thuật toán $O(N^2)$ `dp[i] = max_len` bị TLE. Ta cần đảo vai trò: đưa độ dài len lên làm chỉ số.
- **Bậc 2 (Ý tưởng đảo nhãn):** Gọi `tail[len]` là **phần tử kết thúc nhỏ nhất** của một dãy con tăng có độ dài len .
- **Bậc 3 (Chặt nhị phân):** Mảng `tail` luôn tăng dần. Với mỗi $x \in A$, dùng `std::lower_bound` tìm vị trí đầu tiên $\geq x$ và ghi đè phần tử đó. Đáp án là kích thước của mảng `tail`. Độ phức tạp $O(N \log N)$.

Bài 3: [Codeforces 1042B - Vitamins](#)

- **Nguồn:** [Codeforces Round 510 \(Div. 2\)](#).
- **Giới hạn:** $N \leq 1000$, giá mỗi chai $\leq 10^5$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Có N loại nước quả, mỗi loại cung cấp một tập hợp vitamin (trong các chữ cái A, B, C) với giá tiền c_i . Tìm chi phí nhỏ nhất để nạp đủ cả 3 vitamin A, B và C.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Tập vitamin chỉ có $2^3 = 8$ trạng thái con. Thay vì tối đa hóa vitamin từ số tiền, ta đảo ngược: tìm chi phí nhỏ nhất để đạt mặt nạ vitamin $mask$.
- **Bậc 2 (Thiết kế trạng thái):** `dp[mask]` = chi phí tối thiểu để sở hữu tập vitamin biểu diễn bởi $mask$ ($mask \in [0, 7]$).
- **Bậc 3 (Chuyển trạng thái):** Khởi tạo `dp[0] = 0`, các ô khác $= \infty$. Với mỗi chai nước có giá c và mặt nạ m : `dp[mask | m] = min(dp[mask | m], dp[mask] + c)`. Đáp án là `dp[7]`.

Bài 4: [Codeforces 1437E - Make It Increasing](#)

- **Nguồn:** [Educational Codeforces Round 97](#).
- **Giới hạn:** $N \leq 5 \times 10^5$, $k \leq N$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Cho mảng A và k vị trí cố định không được phép thay đổi. Cần thay đổi ít phần tử nhất để mảng A trở thành mảng tăng nghiêm ngặt ($A_1 < A_2 < \dots < A_n$).

Gợi ý 3 bậc:

- **Bậc 1 (Chuẩn hóa tăng nghiêm ngặt):** Điều kiện $A_i < A_j \iff A_j - A_i \geq j - i \iff A_j - j \geq A_i - i$. Đặt $B_i = A_i - i$, bài toán trở thành tìm dãy con không giảm dài nhất trên mảng B .
- **Bậc 2 (Chia đoạn độc lập):** Các vị trí cố định chia mảng thành $k + 1$ đoạn độc lập. Trong mỗi đoạn (L, R) , ta cần tìm số lượng phần tử giữ lại nhiều nhất thỏa mãn $B_L \leq B_x \leq B_R$.
- **Bậc 3 (LIS đảo nhãn):** Trong mỗi đoạn, áp dụng thuật toán LIS đảo nhãn $O(M \log M)$ với `std::upper_bound` để tìm dãy con không giảm dài nhất.

2. Cấp Độ 2: Trung Cấp & Lập Lịch / Đồ Thị (Intermediate & Scheduling/Graphs)

Bài 5: [Codeforces 864E - Fire](#)

- **Nguồn:** [Codeforces Round 436 \(Div. 2\)](#).
- **Giới hạn:** $N \leq 100, t_i \leq 20, d_i \leq 2000, p_i \leq 20$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Cứu các món đồ khỏi đám cháy. Món đồ i mất t_i thời gian để cứu, bị cháy rụi tại thời điểm d_i , và mang lại giá trị p_i . Tìm tập đồ vật cứu được có tổng giá trị lớn nhất và in ra thứ tự cứu.

Gợi ý 3 bậc:

- **Bậc 1 (Thứ tự cứu đồ vật):** Nếu cứu một tập đồ vật, ta **bắt buộc phải cứu theo thứ tự deadline d_i tăng dần** (nguyên lý tham lam kết hợp DP).
- **Bậc 2 (Mô hình Knapsack có thứ tự):** Sau khi sắp xếp các đồ vật theo d_i tăng dần, bài toán trở thành biến thể Knapsack: thời gian là tài nguyên, p_i là giá trị.
- **Bậc 3 (Chuyển trạng thái & Truy vết):** $dp[j]$ = giá trị lớn nhất đạt được tại thời điểm j . Với món đồ có (t_i, d_i, p_i) , duyệt ngược j từ $d_i - 1$ về t_i : $dp[j] = \max(dp[j], dp[j - t_i] + p_i)$.

Bài 6: [Codeforces 1221D - Make The Fence Great Again](#)

- **Nguồn:** [Educational Codeforces Round 73](#).
- **Giới hạn:** $q \leq 100$ tests, $\sum n \leq 3 \times 10^5, a_i, b_i \leq 10^9$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Tăng chiều cao các thanh hàng rào a_i với chi phí b_i cho mỗi đơn vị tăng, sao cho không có 2 thanh liền kề nào có chiều cao bằng nhau và tổng chi phí là nhỏ nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Bổ đề giới hạn tăng):** Mỗi thanh hàng rào **chỉ cần tăng tối đa +0, +1 hoặc +2!** (Vì xét 2 thanh hàng xóm trái và phải, chúng chỉ có thể trùng tối đa 2 giá trị chiều cao, do đó trong 3 giá trị $\{a_i, a_i + 1, a_i + 2\}$ luôn tồn tại ít nhất 1 giá trị khác cả 2 hàng xóm).
- **Bậc 2 (Đảo nhãn trạng thái):** Thay vì coi chiều cao là số nguyên bất kỳ (10^9), ta chỉ lưu mức tăng $j \in \{0, 1, 2\}$.
- **Bậc 3 (Chuyển trạng thái $O(1)$):** $dp[i][j]$ = chi phí tối thiểu cho tiền tố i thanh với thanh thứ i tăng thêm j . $dp[i][j] = j * b_i + \min_{k \in \{0,1,2\}, a_{i-1}+k \neq a_i+j} dp[i-1][k]$.

Bài 7: [AtCoder ABC 275 F - Erase Subarrays](#)

- **Nguồn:** [AtCoder Beginner Contest 275 — Problem F](#).
- **Giới hạn:** $N, M \leq 3000, A_i \leq 3000$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Với mỗi giá trị $x \in [1, M]$, hãy tìm số lượng đoạn con liên tiếp **tối thiểu** cần xóa khỏi mảng A sao cho tổng các phần tử còn lại đúng bằng x .

Gợi ý 3 bậc:

- **Bậc 1 (Đào góc nhìn):** Thay vì tìm các đoạn con bị xóa, ta tìm các phần tử **được giữ lại**. Mỗi lần chuyển từ trạng thái "xóa" sang "giữ lại" (hoặc ngược lại) tương ứng với việc bắt đầu/kết thúc một đoạn con bị xóa.
- **Bậc 2 (Thiết kế trạng thái):** $dp[x][state]$:
 - x : Tổng giá trị các phần tử được giữ lại ($0 \leq x \leq M$).
 - $state \in \{0, 1\}$: Phần tử vừa xét trước đó là bị xóa (0) hay được giữ lại (1).
 - Giá trị lưu trữ: Số đoạn con bị xóa nhỏ nhất.
- **Bậc 3 (Chuyển trạng thái):** Khi xét A_i :
 - Nếu chọn A_i (giữ lại): $dp[x + A_i][1] = \min(dp[x][1], dp[x][0])$.
 - Nếu xóa A_i : $dp[x][0] = \min(dp[x][0], dp[x][1] + 1)$ (bắt đầu một đoạn xóa mới).

Bài 8: [Codeforces 437C - The Child and Toy](#)

- **Nguồn:** [Codeforces Round 250 \(Div. 2\)](#).
- **Giới hạn:** $N \leq 1000, M \leq 2000, v_i \leq 10^5$, Thời gian: 1.0s.
- **Tóm tắt bài toán:** Đồ thị có N đỉnh với trọng số v_i . Mỗi bước ta xóa một đỉnh, phải trả chi phí bằng tổng trọng số của các đỉnh kề nó còn lại. Tìm tổng chi phí nhỏ nhất để xóa toàn bộ đồ thị.

Gợi ý 3 bậc:

- **Bậc 1 (Đào góc nhìn theo cạnh):** Thay vì tính chi phí theo từng đỉnh bị xóa, ta hỏi: **Mỗi cạnh** (u, v) đóng góp bao nhiêu vào tổng chi phí?
- **Bậc 2 (Phân tích cạnh):** Cạnh (u, v) sẽ được tính vào chi phí đúng một lần: tại thời điểm đỉnh đầu tiên trong cặp $\{u, v\}$ bị xóa! Chi phí phải trả chính là trọng số của đỉnh còn lại.
- **Bậc 3 (Chiến lược tối ưu):** Để tối thiểu hóa chi phí cho mỗi cạnh, ta luôn ưu tiên xóa đỉnh có trọng số lớn hơn trước. Khi đó, chi phí của cạnh (u, v) luôn bằng $\min(v_u, v_v)$. Tổng chi phí tối thiểu chính là $\sum_{(u,v) \in E} \min(v_u, v_v)$.

3. Cấp Độ 3: Nâng Cao & Phân Bổ Hai Chiều (Advanced & Multi-Resource)

Bài 9: [Codeforces 730J - Bottles](#)

- **Nguồn:** [ACM-ICPC NEERC 2016](#).
- **Giới hạn:** $N \leq 100, a_i \leq b_i \leq 100$, Thời gian: 2.0s.

- **Tóm tắt bài toán:** Có N chai nước, chai thứ i chứa a_i lít nước và có dung tích tối đa b_i . Tìm số chai tối thiểu k để chứa hết toàn bộ lượng nước $\sum a_i$, và thời gian tối thiểu (số lít nước cần di chuyển) để gom nước vào k chai đó.

Gợi ý 3 bậc:

- **Bậc 1 (Bước 1 - Tìm k tham lam):** Sắp xếp dung tích b_i giảm dần. Chọn k chai lớn nhất sao cho tổng dung tích $\geq \sum a_i$.
- **Bậc 2 (Bước 2 - Đảo nhãn tối ưu nước chuyển):** Tối thiểu hóa lượng nước chuyển sang chai khác $\iff$ Tối đa hóa lượng nước ban đầu nằm sẵn trong k chai được chọn.
- **Bậc 3 (DP Ba lô 2 chiều):** $dp[i][j][cap]$ = lượng nước a lớn nhất khi chọn đúng j chai trong số i chai đầu tiên với tổng dung tích là cap .

Bài 10: [Codeforces 19B - Checkout Assistant](#)

- **Nguồn:** [Codeforces Beta Round 19](#).
- **Giới hạn:** $N \leq 2000, t_i \leq 2000, c_i \leq 10^9$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Trong siêu thị, món đồ i mất t_i giây quét mã và có giá c_i . Khi thu ngân đang quét món i , bạn có thể lấy trộm các món đồ khác (mỗi giây trộm được 1 món). Tìm số tiền tối thiểu để lấy được toàn bộ N món đồ.

Gợi ý 3 bậc:

- **Bậc 1 (Mô hình hóa tài nguyên):** Nếu bạn mua món i , bạn trả c_i tiền và "thu được" thời gian quét t_i (trộm được t_i món) cộng với chính món $i \implies$ Tổng số món đồ nhận được là $t_i + 1$.
- **Bậc 2 (Quy về Knapsack đảo nhãn):**
 - "Trọng lượng" của món đồ i là $w_i = t_i + 1$.
 - "Chi phí" là c_i .
 - Cần chọn một tập món đồ sao cho tổng trọng lượng $\geq N$ với **tổng chi phí nhỏ nhất**.
- **Bậc 3 (Chuyển trạng thái):** $dp[j]$ = chi phí tối thiểu để thu được ít nhất j món đồ ($0 \leq j \leq N$).
 $dp[\min(N, j + t_i + 1)] = \min(\dots, dp[j] + c_i)$.

Bài 11: [Codeforces 938D - Buy a Ticket](#)

- **Nguồn:** [Educational Codeforces Round 38](#).
- **Giới hạn:** $N, M \leq 2 \times 10^5, A_i, w_e \leq 10^{12}$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Có N thành phố. Ở thành phố u có buổi hòa nhạc giá vé A_u . Đi lại giữa u và v tốn vé tàu $w(u, v)$. Với mỗi thành phố u , tìm chi phí nhỏ nhất để một người xuất phát từ u đi đến một thành phố bất kỳ xem hòa nhạc rồi quay về u .

Gợi ý 3 bậc:

- **Bậc 1 (Đảo hướng tư duy):** Thay vì tìm đường đi tốt nhất xuất phát từ từng đỉnh u , ta nhìn từ góc độ đỉnh diễn ra buổi hòa nhạc.

- **Bậc 2 (Mô hình Siêu nguồn - Super Source):** Tạo một đỉnh ảo 0. Nối cạnh có hướng từ đỉnh ảo 0 đến mọi đỉnh u với trọng số là giá vé A_u .
- **Bậc 3 (Dijkstra đa nguồn đảo nhãn):** Cạnh hai chiều (u, v) có trọng số là $2 \times w(u, v)$ (đi và về). Chạy thuật toán Dijkstra duy nhất xuất phát từ đỉnh ảo 0. Khoảng cách ngắn nhất từ 0 tới u chính là đáp án cho thành phố u !

Bài 12: [AtCoder ABC 221 G - Jumping Frog](#)

- **Nguồn:** [AtCoder Beginner Contest 221](#).
- **Giới hạn:** $N \leq 2000, \sum D_i \leq 3.6 \times 10^6, |A|, |B| \leq \sum D_i$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Bắt đầu từ $(0, 0)$, bước thứ i nhảy đúng khoảng cách D_i theo 1 trong 4 hướng (L, R, U, D). Hỏi có thể đến được (A, B) sau N bước không? Nếu được, hãy dựng đường đi.

Gợi ý 3 bậc:

- **Bậc 1 (Xoay hệ trục tọa độ 45°):** Biến đổi tọa độ Manhattan thành Chebyshev: $(X, Y) = (A + B, A - B)$. Mỗi bước nhảy theo 4 hướng ban đầu biến thành việc độc lập thay đổi $X \pm D_i$ và $Y \pm D_i$!
- **Bậc 2 (Quy về Subset Sum):** Cần chọn dấu $+$ hoặc $-$ cho mỗi D_i để tổng bằng X (và Y). Tương đương chọn tập con có tổng bằng $\frac{\sum D_i + X}{2}$.
- **Bậc 3 (Tối ưu Bitset đảo nhãn):** Sử dụng `std::bitset<3600005>` để kiểm tra khả năng tạo tổng trong $O(\frac{N \cdot \sum D}{64})$, sau đó truy vết lại hướng nhảy.

4. Cấp Độ 4: Chinh Phục & Tối Ưu Đỉnh Cao (Mastery & Extreme Applications)

Bài 13: [Luogu P3188 - 【HNOI2007】 邦邦的大背包](#)

- **Nguồn:** [HNOI 2007 / Luogu P3188](#).
- **Giới hạn:** $N \leq 100, W \leq 10^9, w_i = a_i \cdot 2^{b_i}$ với $a_i \leq 10, b_i \leq 30$. Thời gian: 1.0s.
- **Tóm tắt bài toán:** Giải bài toán Ba lô 0-1 với W cực lớn lên tới 10^9 , nhưng trọng lượng các đồ vật có dạng đặc biệt $a_i \cdot 2^{b_i}$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhóm theo lũy thừa 2):** Phân chia các đồ vật vào 31 nhóm theo số mũ b_i . Nhóm b chỉ chứa các vật có trọng lượng $a_i \cdot 2^b$ với $a_i \leq 10$.
- **Bậc 2 (Đảo nhãn theo từng bit):** Biểu diễn W dưới dạng nhị phân. Tại tầng b , sau khi chọn các đồ vật ở tầng b , phần dư tích dư thừa được nhân đôi khi chuyển xuống tầng $b - 1$.
- **Bậc 3 (Quy hoạch động theo cây bit):** `dp[b][rem]` = giá trị lớn nhất khi xét từ bit 30 xuống bit b , với dư tích dư của tầng này là $rem \cdot 2^b$.

Bài 14: [Codeforces 1442D - Sum of Paths](#)

- **Nguồn:** [Codeforces Round 681 \(Div. 1\)](#).
- **Giới hạn:** $N, K \leq 3000$, các mảng có tổng độ dài $\leq 10^6$, các phần tử trong mỗi mảng tăng dần. Thời gian: 1.5s.
- **Tóm tắt bài toán:** Có N mảng số nguyên không giảm. Cần chọn ra tổng cộng đúng K phần tử từ đầu các mảng sao cho tổng giá trị là lớn nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Bổ đề Knapsack cực trị):** Vì các phần tử trong mỗi mảng đều tăng dần, hàm giá trị theo số phần tử lấy là **hàm lồi**. Do tính chất lồi, trong phương án tối ưu, **có tối đa 1 mảng được lấy một phần lờ** lửng, tất cả $N - 1$ mảng còn lại hoặc được lấy toàn bộ hoặc không được lấy phần tử nào!
- **Bậc 2 (Chia để trị loại trừ):** Ta duyệt qua mảng i đóng vai trò là mảng duy nhất được lấy một phần. Khi đó, ta cần giải bài toán Knapsack với $N - 1$ mảng còn lại (được coi như các đồ vật nguyên khối có trọng lượng là độ dài mảng và giá trị là tổng mảng).
- **Bậc 3 (D&C Knapsack):** Dùng kỹ thuật chia để trị (Divide and Conquer) để tính mảng Knapsack khi loại trừ lần lượt từng mảng i trong $O(N \cdot K \log N)$.

Bài 15: [AtCoder AGC 020 C - Median Sum](#)

- **Nguồn:** [AtCoder Grand Contest 020](#).
- **Giới hạn:** $N \leq 2000, A_i \leq 2000 \implies \sum A_i \leq 4 \times 10^6$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Xét tất cả $2^N - 1$ tập con không rỗng của A . Tính tổng của mỗi tập con, sắp xếp $2^N - 1$ tổng này theo thứ tự tăng dần. Tìm phần tử trung vị (phần tử thứ $\lceil \frac{2^N - 1}{2} \rceil$).

Gợi ý 3 bậc:

- **Bậc 1 (Tính đối ngẫu bù trừ):** Mỗi tập con S đều có tập bù $S' = U \setminus S$. Tổng của chúng thỏa mãn: $\text{sum}(S) + \text{sum}(S') = \sum_{i=1}^N A_i = S_{\text{all}}$.
- **Bậc 2 (Đối xứng quanh điểm giữa):** Phép ghép cặp (S, S') chia 2^N tập con thành các cặp có tổng đối xứng nhau qua $\frac{S_{\text{all}}}{2}$. Do đó, trung vị chính là **tổng tập con nhỏ nhất mà $\geq \lceil \frac{S_{\text{all}}}{2} \rceil$** !
- **Bậc 3 (Bitset đảo nhãn):** Dùng `std::bitset<4000005>` để tìm tất cả các tổng tập con có thể tạo ra trong $O(\frac{N \cdot S_{\text{all}}}{64}) \approx 1.2 \times 10^8$ phép tính, sau đó tìm bit 1 đầu tiên từ vị trí $\lceil \frac{S_{\text{all}}}{2} \rceil$.

Bài 16: [Codeforces 1083E - The Fair Nut and Rectangles](#)

- **Nguồn:** [Codeforces Round 526 \(Div. 1\)](#).
- **Giới hạn:** $N \leq 10^6, 1 \leq x_i, y_i \leq 10^9, 0 \leq a_i < x_i \cdot y_i$, Thời gian: 2.0s.
- **Tóm tắt bài toán:** Cho N hình chữ nhật có góc trái-dưới tại $(0, 0)$, góc phải-trên tại (x_i, y_i) , không có hình nào chứa hoàn toàn hình khác. Chọn một tập hình chữ nhật sao cho diện tích phần hợp trừ đi tổng chi phí a_i là lớn nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Sắp xếp đảo góc nhìn):** Sắp xếp các hình chữ nhật theo x_i tăng dần (đồng nghĩa với y_i giảm dần). Khi đó diện tích phần giao nhau giữa hai hình liên tiếp $j < i$ chỉ đơn giản là $x_j \cdot y_i$.
- **Bậc 2 (Công thức DP):** $dp[i] = x_i y_i - a_i + \max_{0 \leq j < i} (dp[j] - x_j y_i)$.
- **Bậc 3 (Tối ưu hóa đường bao lồi - CHT):** Biểu thức có dạng đường thẳng $y = m \cdot x + c$ với hệ số góc $m = -x_j$ đơn điệu giảm và truy vấn tại y_i đơn điệu giảm. Áp dụng Convex Hull Trick duy trì bằng Deque để tính mỗi bước trong $O(1)$, tổng thời gian $O(N \log N)$ (do sắp xếp).



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.